Ders 1 Genişletilmiş Notu: Tanışma, Dersin İşleyişi, Ayrık Matematiğe Giriş ve Önermeler Mantığı

Ders 1 Genişletilmiş Notu: Tanışma, Dersin İşleyişi, Ayrık Matematiğe Giriş ve Önermeler Mantığı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yaklaşımı: Algoritma, Terimler ve Kaynak
- 2. Dönemin Konu Başlıkları
- 3. Ayrık Matematik Nedir?
- 4. Ayrık Matematikle Çözülen Problemler
- 5. Dersin Amacı ve Algoritmik Düşünme
- 6. Ayrık Matematik Bir Geçiş Dersidir
- 7. Kombinatoryal Problem Türleri
- 8. Bir Planlama Problemi: PERT
- 9. Bir Eşleştirme Problemi: Pilotlar ve Uçuşlar
- 10. Mantık ve Önermeler
- 11. Bağlaçlar ve Gösterimleri
- 12. AND (Ve) ve Doğruluk Tablosunun Kuruluşu
- 13. OR (Veya)
- 14. XOR (Dışlayan Veya)
- 15. NOT (Değil)
- 16. Birleştirilmiş Önermeler
- 17. Koşullu Önerme
- 18. Örnek Sorular

1. Dersin Yaklaşımı: Algoritma, Terimler ve Kaynak

1.1 Algoritmanın önemi

Hoca, dersin asıl amacını en başta açıkça söyledi: bir bilgisayar mühendisliği öğrencisi için algoritma her şeyin temelidir. Algoritmayı geliştirmenin tek yolu kod yazmaktır. Kod yazmak için de önünüzde problemler olmalıdır. Bu yüzden ders boyunca bazı konularda “bunun algoritmasını yazın” denilen küçük problemler verilecek. Bu problemler büyük değildir, ama çözmeye çalışan kazanır.

Hoca, Veri Yapıları ve Algoritma Analizi gibi derslerde çok sayıda öğrencinin kalmasının sebebini de buna bağladı: algoritma mantığı oturmamıştır. Seçmeli derslerin ve ara projelerin hepsi projeye dayalıdır, bu yüzden üçüncü sınıfa gelindiğinde algoritma ile ilgili sorunların çözülmüş olması gerekir.

Hoca kendi öğrencilik deneyimini de anlattı. Birinci sınıfta algoritmayı anlamadığı için bölümü bırakmayı düşünmüş. Sonra sürekli çalışarak bir gün “ışığın yandığını” görmüş. Hocaya göre bu ışık bir anda yanmaz:

- Akışı çiz.
- Programı yaz.

- Çalıştır.
- Hatayı bul, düzelt.

“Kimseye tepeden zembille bu özellikler inmiyor. Çalışarak bunları elde ediyoruz.”

1.2 Terimlerin İngilizcesi ve Türkçesi

Hoca, terminolojiye girmiş kelimeleri hem İngilizce hem Türkçe vereceğini söyledi. Bir konuyu araştırırken Türkçe kaynak da bulunur, ama İngilizce kaynak çok daha fazladır. Ayrıca bazı Türkçe karşılıklar kulağa yabancı gelir. Hocanın örnekleri:

- *Computer* aslında “hesaplayan” demektir. “Bilgisayar” kelimesi hayatımıza ilk böyle girdiği için onu sorgulamıyoruz bile.
- Lojik derslerinde sık geçen *port* kelimesi “iskele” diye çevrilir. “İskele” denince akla gemilerin yanaştığı yer gelir; anlam tam örtüşmez.

Bu yüzden terimler her iki dilde birlikte verilecek.

1.3 Ders kitabı

Slaytların büyük çoğunluğu, hocanın tahminiyle yaklaşık %95’i, Rosen’in *Discrete Mathematics and Its Applications* kitabından hazırlanmıştır. Bazı slaytlar kitabın slaytlarından Türkçeye çevrilmiş, bazıları sonradan eklenmiştir. Bu kitap, ayrık matematiğin okutulduğu hemen her üniversitede ders kitabı olarak kullanılır ve her yeni baskıda sayfa sayısı artar.

Hoca başka bir kitap önermedi. Gerekçesi şuydu: Ayrık matematik kitaplarının kapsamları birbirinden farklıdır. Bir kitapta Rosen’deki konuların bir kısmı, başka bir kitapta başka bir kısmı vardır. Derste anlatılan konuların hepsi tek bir başka kitapta yoktur. Konu başlıklarıyla internette arama yapılırsa yüzlerce kaynak bulunur.

Bölüm sonundaki kaynaklar listesinde Rosen’in iki kitabı ve “Ayrık İşlemsel Yapılar” adlı bir ders notu da vardı. Hoca bu vesileyle bir ilkeyi de vurguladı: Bir kaynaktan yararlanıldıysa mutlaka referans olarak verilmelidir.

2. Dönemin Konu Başlıkları

Hoca dönem boyunca işlenecek bölümleri tek tek tanıttı.

- **Ayrık matematiğe giriş:** Matematiğin “ayrık” olması ne demektir?
- **Mantık ve ispatlar (logic and proofs):** Önermeler, koşullu ve koşulsuz önermeler, niceleyiciler (quantifiers), matematiksel bir ifadenin nasıl ispatlanacağı ve ispat yöntemleri.
- **Boole cebri ve kombinasyonel devreler (combinational circuits):** Hoca bu bölümü sayısal (lojik) devreler dersinin bir girişi olarak gördüğünü, o derste oturmayan kısımların burada ikinci kez görülmesi için konduğunu söyledi. Karnaugh diyagramları bu bölümdedir ve video ile anlatılacaktır.
- **Matematiğin dili:** Matematiği matematik yapan ifadeler: matrisler, kümeler, diziler, sayı sistemleri, bağıntılar (ilişkiler). Bu bölüm ve sonraki bölüm bu dönem yeni eklenmiştir.
- **Sayı teorisi:** Hocaya göre bu konular bilgisayar bilimlerine girişte gösterilmelidir.
- **Algoritmalar, karmaşıklık ve asimptotik gösterim.**
- **Graf teorisi ve ağaçlar.**
- **Kromatik polinomlar.**
- **Otomatlar, gramerler ve diller.**

2.1 Karmaşıklık neden önemli?

Hoca algoritma karmaşıklığını bir örnekle anlattı. Bir problemi biri iki `if` ile, öbürü beş `if` ile çözmüş olsun. İkisi de sonuca ulaşır. Ama beş `if` kullanan çözümün karmaşıklığı büyük olasılıkla daha yüksek çıkacaktır. Bir algoritma yazılıyorsa iyi optimize edilmelidir: karmaşıklığı en düşük, çalışma süresi (runtime) en kısa olmalıdır.

“Elinizi vicdanınıza koyun”: Aynı işi yapan iki programdan biri 5 milisaniyede, öbürü bir dakikada çalışıyorsa, herkes ilkini seçer.

2.2 Graflar ve ağaçlar

> **Hocanın vurgusu:** Graf, ağaç ve otomatlar, ilerideki birçok dersin temelini oluşturur. O dersler bu konuları bildiğiniz varsayılarak kurulur. Graf teorisi ve ağaçları çok dikkatli dinleyin.

Bir graf, düğümlerden ve düğümleri birbirine bağlayan kenarlardan oluşur. Graf teorisinde sorulan sorulara örnekler:

- Bütün düğümleri minimum maliyetle gezmek istiyorum. Başladığım düğümden bütün düğümleri gezip aynı yere en kısa yolla nasıl dönerim?
- Görsel olarak karmaşık bir grafı nasıl sadeleştiririm? Bazı graflara belirli adlar verilir. Örneğin bir graf Hamilton grafıysa, o adı vermek grafın taşıdığı bazı özellikleri de söylemiş olur.

Ağaçlar da çok önemlidir; lisans, yüksek lisans ya da doktora fark etmeksizin kullanılan algoritmaların birçoğunda ağaçlar vardır. Ağaç aslında bir veri yapısıdır: Birçok veri ağaç üzerinde tutulur. Ağaçtaki veriler nasıl aranır, nasıl okunur, nasıl sıralanır, bunlar bu bölümde görülecek.

Hoca algoritmalardaki ağacı bahçedeki ağaca benzetti, ama ters çevrilmiş haliyle: Ağacı kökünden çıkarıp ters çevirin. Kök en üsttedir ve ağaç aşağıya doğru dallanır. Bir aile ağacı çizimi gibi düşünülebilir. Ağaçlar grafların özel bir halidir; bu yüzden ikisi aynı bölümde verilir.

Kromatik polinomlar karışık bir konudur. Adım adım (step by step) anlatılması gerektiği için bunun için de video çekilecektir.

2.3 Otomatlar, gramerler ve diller

Bu konular derleyici tasarımı (compiler design) dersinin olmazsa olmaz malzemeleridir. Bunları bilmeden o dersteki bazı kavramlar anlaşılamayabilir. Derste bir öğrencinin hatırlatmasıyla, bu konuların Hesaplama Kuramı dersinin de girişi olduğu söylendi.

Hoca dersi bir dizinin fragmanına benzetti: Ayrık matematiğin her konu başlığı, lisansta alınacak zorunlu ya da seçmeli bir dersin girişini oluşturur. Bu ders o derslere bir ön hazırlıktır.

3. Ayrık Matematik Nedir?

Dersin eski adı “Discrete Matematik” idi, sonra “Ayrık Matematik” oldu. Peki matematiğin “ayrık” olması ne demektir?

3.1 Ses örneği: sürekli işaretten ayrık örneklere

Hoca bunu bir ses kaydı ile anlattı. Hocanın sesini bir WAV dosyası olarak kaydettiğinizi ve bir araçla açtığınızı düşünün. Gördüğünüz şey sürekli bir ses dalgasıdır.

Bilgisayar bilimlerinde (computer science) sürekli işaretlerle, sürekli verilerle çalışılamaz. Sistem buna izin vermez, çünkü bilgisayar 1 ve 0 mantığıyla çalışır. Bu yüzden:

- Ses işaretinden belirli t aralıklarında örnekler (samples) alınır.
- İşlem bu örnekler üzerinde yapılır.
- Örnekler saklanır. Gerekirse bir fonksiyonla birbirine bağlanarak sesin orijinal hali yeniden elde edilir.

Hoca bunun Fourier serileriyle yapılabileceğini söyledi. Fourier serisi ayrık matematiğin değil, sürekli matematiğin konusudur. Ama elimizde Fourier serisine açılmış bir işaret varsa, bilgisayarda yapacağımız şey yine ondan belirli aralıklarla örnekler alıp onları işlemektir.

Matematiğin “ayrık” diye adlandırılmasının anlamı budur. Sürekli bir girdi üzerinde işlem yapılamaz. İşleyebilmek için tek tek, birbirinden ayrı verilere ihtiyaç vardır.

> **Ek bilgi (derste anlatılmadı):** Sinyal işlemede örnekleme teoremi (Nyquist-Shannon), sinyaldeki en yüksek frekansın en az iki katı hızla örnek alınırsa bant sınırlı bir sinyalin örneklerden tam olarak geri elde edilebileceğini söyler. Derste

"örneklerden orijinal ses yeniden elde edilir" denirken kastedilen durum budur. Bu teorem sürekli matematiğe aittir; ayrık matematik dersinin konusu değildir.

3.2 Tanım

Daha biçimsel söylersek: **Ayrık matematik, matematiğin ayrık yapılarla ilgilenen, süreklilik içermeyen konularını kapsayan dalıdır.** Klasik matematik sürekli nesnelerle de ilgilenirken ayrık matematik ayrık nesnelerle ilgilenir.

3.3 Ayrık nesne örnekleri

- **Tam sayılar.** Hoca burada 1 ile 2 arasını örnek verdi. Bu aralıkta sonsuz sayıda sayı vardır. Bilgisayarda bunları ancak 1.1, 1.11, 1.111, 1.1111 gibi değerlerle, aralığı gittikçe daraltarak temsil edebiliriz. Ama alınan her değer yine ayrıktır.
- **Bir bilgisayar programının yürüttüğü işlem adımları.** Bir kod tek bir bütün olarak yazılamaz; parça parça yazılır.
- **Bir harita üzerinde A noktasından B noktasına giden yollar.** İstanbul'dan Gebze'ye gidilirken insan ışınlanmaz: Kartal, Pendik gibi duraklardan geçilir. Geçilen her nokta ayrık bir noktadır. Aradaki semtler de eklenirse veri artar, ama yine ayrıktır.

Ayrık matematik, bilgisayar bilimlerindeki birçok ders için gereken matematiksel altyapıyı sağlar. Kısacası: Süreklilik yok, her şey parça parça.

4. Ayrık Matematikle Çözülen Problemler

Hoca, ayrık matematikle çözülen problem türlerini slayttan sıraladı:

- **Şifre sayma:** Belirli kurallara göre bir şifre kaç yolla seçilebilir? Örneğin 8 karakterli, en az bir büyük harf ve en az bir özel karakter içeren kaç farklı şifre yazılabilir?
- **İnternet adresleri:** Bir internet adresinin belirli bir formatı vardır. Bu format ve ASCII karakterleri kullanılarak kaç farklı geçerli adres oluşturulabilir?
- **Piyango:** Belirli bir piyangoyu kazanma olasılığı nedir?
- **Ağ bağlantısı:** Bir ağdaki iki bilgisayar arasında bağlantı var mı? Varsa kaç farklı bağlantı var?
- **İstenmeyen e-posta (spam):** Gelen bir mesajın spam olup olmadığını bulan bir uygulama nasıl yazılır? Önce spam bir mesajın sahip olduğu özellikler çıkarılır. Gelen mesajda bu özellikler varsa mesaj spam sayılır. Çıkarılan özellikler tek tek, ayrık verilerdir. Biri 5, bir başkası 10 özellik çıkarabilir; sonuçlar da farklı olur.
- **Şifreleme:** Bir mesaj, istenmeyen bir alıcı okuyamasın diye nasıl şifrelenir?
- **Toplama devresi:** İki tam sayıyı toplayan bir devre nasıl kurulur?
- **En kısa yol:** Bir ulaşım sisteminde iki şehir arasındaki en kısa yol nasıl bulunur? İstanbul'dan Ankara'ya klasik yoldan gidilebilir, Çanakkale ve İzmir üzerinden de gidilebilir. Bu, graflar konusunda görülecek.
- **En kısa tur:** Bir grup şehrin her birini yalnız bir kez ziyaret edip başlangıç şehrine dönen en kısa tur nasıl bulunur? Hocanın örneği, bir ülkeye giden turistik gezilerdir: Sizi bir noktadan alır, farklı güzergâhlardan dolaştırır ve başladığınız noktaya geri getirir.
- **Mantık yürütme:** Bir bilgisayarın mantık yürütebilmesi için cümleler nasıl temsil edilir?
- **İspat:** Sonsuz sayıda asal sayı olduğu nasıl kanıtlanır?
- **Sıralama:** Bir tam sayı listesi artan sırada nasıl sıralanır? Kaç adım gerekir? Bir sıralama algoritmasının listeyi her zaman doğru sıraladığı nasıl kanıtlanır?

Hoca, bu soruların benzerlerinin Algoritma Analizi ve Veri Yapıları derslerinde ödev ya da sınav sorusu olarak karşınıza çıkacağını söyledi. Hepsinde ortak olan şey, bir bilgisayar bilimcisinin problemi ayrık olarak düşünebilmesi, algoritmasını kurabilmesi ve kodlayabilmesidir. Böyle düşünülemezse problemler ancak teoride çözülür, bilgisayar ortamında çözülemez.

5. Dersin Amacı ve Algoritmik Düşünme

Slayttaki amaçlar şöyle özetlendi:

- **Matematiksel akıl yürütme (mathematical reasoning):** Argümanlar ortaya koymak, bir hipotez atmak ve bu hipotezi ispatlamaya çalışmak.
- **Kombinatoryal analiz (combinatorial analysis):** Farklı türdeki nesneleri sayma teknikleri.
- **Ayrık yapılar (discrete structures):** Kümeler, permütasyonlar, ilişkiler, graflar, ağaçlar ve sonlu durum makineleri (finite state machines).
- **Algoritmik düşünce (algorithmic thinking):** Mezun olana kadar sahip olunması gereken beceri.

Hoca, dış paydaşlardan gelen geri dönüşlerde bölüm mezunlarının algoritmik düşüncesinin güçlü bulunduğunu, bunu övünerek söyleyebileceğini anlattı. Birçok problemi çözmenin tek yolu algoritmayı belirlemektir.

5.1 Terzi benzetmesi: Koda hemen başlamayın

> **Hocanın vurgusu:** Bilgisayarın başına oturup hemen kod yazmak büyük bir hatadır. Önce akış çizin, analizi akış üzerinde yapın, kendinizden emin olunca kodlayın.

Hoca son yıllarda öğrencilerde gördüğü bir alışkanlıktan yakındı: Havaya bakıp algoritmayı kafada düşünmek, sonra oturup doğrudan kodlamak. Bir terzi makası alıp kumaşı hemen kesmez; önce ölçü alır, bir patron çıkarır. Bir makine üreten de parçaları hemen birleştirmez; neyi neyle birleştirirse daha verimli çalışacağını düşünür.

Akışın ince ayrıntılarla çizilmesi gerekmez. Bir blok şeması yeterlidir: Şurada bir modül olacak, bu modülde şunlar bulunacak, sonra şu, sonra bu.

5.2 Uç durumlar

> **Hocanın vurgusu:** Uç durumları (edge cases) kesinlikle düşünün; hiç göz ardı etmeyin.

Uç durum, programı yazan kişinin düşünmediği, aklına bile gelmeyen durumdur. Hangi girdi verilirse algoritmanın başarısız (fail) olacağı önceden bulunmalı ve algoritma buna göre kurulmalıdır. Algoritma gerçek hayatta kullanıldığında bu durumlar mutlaka karşınıza çıkar.

5.3 Uygulamalar ve modelleme

Ayrık matematik kavramları yalnız hesaplama problemleri için kullanılmaz. Kimya, biyoloji, dilbilim, coğrafya, işletme, iktisat gibi birçok alandaki problemlerin çözümünde de kullanılır. Hoca disiplinler arası çalışmanın değerini vurguladı: Bir projeye farklı alanlardan bilgi katılırsa projenin kullanım alanı artar. Başka bir alanda yüksek lisans yapan bir bilgisayar mühendisi, algoritma bilgisiyle bir uygulama geliştirdiğinde o alandan gelenlerin önüne geçer.

6. Ayrık Matematik Bir Geçiş Dersidir

Arada verilen kısa molanın ardından hoca, ayrık matematiğin bir geçiş dersi olduğunu hatırlattı. Slaytta ayrık matematiğin giriş oluşturduğu bilgisayar bilimi dersleri vardı: bilgisayar mimarisi, veri yapıları, algoritmalar, programlama dilleri, derleyiciler, bilgisayar güvenliği, veri tabanı, yapay zekâ, ağlar, bilgisayar grafiği, oyun tasarımı, hesaplama teorisi ve diğerleri.

Ayrık matematik dersi başka bölümlerde de, örneğin matematik mühendisliğinde, okutulur. Her bölüm dersi kendi dersleriyle ilişkili konulara göre şekillendirir. Ders diğer disiplinlerle de ilişkilidir. Hoca felsefeyle ilişkiyi mantık üzerinden kurdu: Mantık hem felsefenin hem ayrık matematiğin konusudur.

7. Kombinatoryal Problem Türleri

Hoca, kombinatoryal problemlere ve tekniklere giriş olarak slayttan dört örnek okudu. Bu örnekler, problemlerin hangi türden olduğunu göstermek içindi; derste çözülmediler.

7.1 Varlık problemi: tenis fikstürü

Dört evli çift her pazar akşamı iki kortta karışık çiftler tenis maçı yapıyor. İki saatlik oyun süresinde her yarım saatte bir eşlerini ve rakiplerini değiştiriyorlar. Her erkeğin her kadınla en az bir kez birlikte ve en az bir kez karşı karşıya oynadığı, ayrıca her erkeğin diğer her erkeğe karşı en az bir kez oynadığı bir fikstür var mı?

Hoca bunun oldukça zor bir problem olduğunu ve bir **varlık belirleme (existence)** problemi olduğunu söyledi. Soru "nasıl" ya da "kaç tane" değil, "böyle bir düzen var mı?" sorusudur. Çözüm denemesi için 18. bölüme bakın.

7.2 Sayma problemi: yatırım kulübü

Altı kişilik bir yatırım kulübü var. Her yıl başkan ve sayman pozisyonlarını dönüşümlü olarak değiştirmek istiyorlar. Aynı kişilerin aynı pozisyonlara yeniden gelmesi için kaç yıl geçmelidir?

Bu bir **sayma (counting)** problemidir. Hocaya göre matematikteki sayma konusunu bilmeyen bu soruyu çözemez. Çözüm 18. bölümdedir.

7.3 Optimizasyon problemi: iş atama

Bir iş yerinde üç çalışan var ve saat ücretleri sırasıyla 10, 12 ve 15 birimdir (altyazıda "10 milyon, 12 milyon, 15 milyon"). Patron bu üç kişiye üç iş verecek. Slayttaki tabloda her kişinin her işi kaç günde yaptığı yazıyor. Örneğin birinci işi bir çalışan 7,5 günde, başka bir çalışan 6 günde yapıyor. Amaç, işleri toplam maliyet en az olacak biçimde dağıtmaktır.

Bu bir **optimizasyon (optimization)** problemidir. Hoca, optimizasyon problemleriyle endüstri mühendislerinin daha çok ilgilendiğini, ama bunun bir bilgisayar mühendisi için aynı zamanda algoritmik bir problem olduğunu söyledi.

Hoca buna benzer bir örnek daha verdi: 5 derslik, 50 ders ve 12 öğretim üyesi var. Dersler dersliklere öyle yerleştirilecek ki hiçbir derslik boş kalmasın ve dersler hocalara eşit dağıtılsın. Kısıtlar arttıkça bu tür problemler yönetim için çok zorlaşır.

8. Bir Planlama Problemi: PERT

8.1 Problem

Büyük bir mağaza bayram indirimi için 8 sayfalık bir ilan göndermeyi planlıyor. İlan bayramdan en az 10 gün önce postalanmalı. Bunun için birçok işin yapılması ve önce bazı kararların alınması gerekiyor. Bütün işler için yalnız 30 gün var.

Slaytta A'dan K'ya kadar etiketlenmiş 11 işin süreleri bir tabloda verilmişti. İşler tek tek, arka arkaya yapılırsa toplam süre 37 gündür. Hoca bunu sesli toplayarak kontrol etti (altyazıda "12, 17, 20, 23, 26, 30, 34, 37" diye geçen ara toplamlar). 37 gün, 30 günlük sınırı aşar. Daha kısa sürede biterse işveren için daha da iyidir. Soru: İşler nasıl düzenlenmeli ki 30 günü aşmasın?

8.2 Graf modeli

Hoca şu ilkeyi slayttan okudu: Bazı durumlarda bilgiyi grafik olarak göstermek çok daha anlaşılır olur.

- Her iş bir düğüm (nokta) ile gösterilir.
- Bir işin yapılabilmesi için başka bir işin bitmesi gerekiyorsa, iki düğüm bir okla birleştirilir.
- Dairelerin içindeki sayılar, o işin kaç günde yapılacağıdır.

Öncelik ilişkileri şöyledir: A ve B bağımsız ilk işlerdir. C için A ve B'nin bitmesi gerekir. D, E ve G için C gerekir. H için G, F için D ve E, I için F, J için H ve I gerekir. En son tamamlanan iş K'dir ve J'ye bağlıdır.

8.3 Adım adım çözüm

Kural: Bir işin bitiş zamanı, kendisinden önce bitmesi gereken işlerin **en geç** bitiş zamanına kendi süresinin eklenmesiyle bulunur. Önceki işlerden erken biten olsa bile, en geç biteni beklemek gerekir.

İş	Süre (gün)	Önce bitmesi gereken	En erken bitiş
A	3	-	3
B	2	-	2
C	2	A, B	$\max(3,2)+2=5$
D	4	C	$5+4=9$
E	3	C	$5+3=8$
G	3	C	$5+3=8$
H	1	G	$8+1=9$
F	2	D, E	$\max(9,8)+2=11$
I	5	F	$11+5=16$
J	2	H, I	$\max(9,16)+2=18$
K	10	J	$18+10=28$

Hocanın anlatımındaki sıra şöyleydi:

- C için A ve B'nin bitmesi gerekir; ikisinin bitmesi için en çok 3 gün gerekir. C'nin kendisi 2 gün sürer. C 5. günde biter.
- G, D ve E, C'den sonra başlar: G $5+3=8$, D $5+4=9$, E $5+3=8$.
- H, G'den sonra: $8+1=9$.
- F için D (9) ve E (8) beklenir: $9+2=11$.
- I, F'den sonra: $11+5=16$.
- J için H (9) ve I (16) beklenir. Hoca burada "dikkatli olun" dedi: H tarafından gelinse $9+2=11$ olurdu, ama I tarafından gelen 16 daha uzundur. En uzununu alırız: $16+2=18$.
- K 10 gün sürer: $18+10=28$.

Sonuç: İşler bu planla **28 günde** tamamlanır. Sıralı planın 37 gününden çok daha kısadır ve 30 günlük sınırın içindedir.

8.4 Kritik yol

Slaytta bazı kenarlar kırmızıyla çizilmişti. İşin tamamlandığı K düğümünden geriye doğru gidilirse: K, J'ye; J, I'ya (çünkü 16, H'nin 9'undan büyüktür); I, F'ye; F, D'ye (9, E'nin 8'inden büyüktür); D, C'ye; C de A'ya (3, B'nin 2'sinden büyüktür) bağlıdır. Bu yol **kritik yol (critical path)** olarak adlandırılır:

$$A \to C \to D \to F \to I \to J \to K, \qquad 3+2+4+2+5+2+10=28$$

Kritik yol, her birleşme noktasında en uzun sürenin seçildiği yoldur.

Bu yöntem **PERT** (Program Evaluation and Review Technique) olarak bilinir. Hoca, aslında PERT'in algoritmasını göstermiş olduğunu, bunu kodlayan kişinin PERT'i kodlamış olacağını söyledi. Merak eden PERT algoritmasının ayrıntılarını inceleyebilir.

> **Ek bilgi (derste anlatılmadı):** Kritik yol üzerindeki işlerin boşluğu (slack) sıfırdır; bu işlerden herhangi birindeki gecikme projenin tamamını aynı miktarda geciktirir. Kritik yol dışındaki bir iş ise, kendi boşluğu kadar gecikebilir. Örneğin bu tabloda H işi 9. günde biter ama J'nin başlaması için 16. gün beklenir.

8.5 Problemi ayrık düşünmek

> **Hocanın vurgusu:** Problemler bize metin olarak verilir. Bir şeyi ayrık düşünebilmek, onu parçalara ayırabilmek demektir. Burada parçalara ayırma, verileri bir grafa dönüştürmek oldu. Graf kurulduktan sonra kodlamak çok kolaydır.

Hoca, bir probleme “paldır küldür” girilmemesi gerektiğini tekrarladı. Önce en iyi nasıl çözüleceği düşünülmelidir. Peki graf fikri insanın aklına nereden gelir? Graf konusunu hiç görmeyen birinin aklına bu gelmez; gelse bile bir buluş yaptığını sanır, oysa o buluş çoktan yapılmıştır. Dersin amacı da budur: Bugün kullanmadığınız küçük bir bilgi, bir iki yıl sonra bir problemle karşılaşınca “buna benzer bir şey görmüştüm” dedirtir. Hiç görülmemiş bir şeyin araştırılması bile mümkün olmaz, çünkü varlığından haberiniz yoktur.

9. Bir Eşleştirme Problemi: Pilotlar ve Uçuşlar

Bir havayolu şirketinin pazartesi sabahları Ankara’dan 7 farklı ülkeye uçuşu var. 7 pilotu var: $P_1, \dots, P_7$. Her pilot her ülkeye uçmak istemiyor; her birinin uçmak istediği ülkeler belli. Örneğin bir pilot Libya, Suriye, Brezilya ve Danimarka’ya uçabileceğini söylüyor. Planlamacı, her uçuşa bir pilot koyacak ve pilotların isteklerini dikkate alacak.

Bu problem metin olarak verildiğinde takip etmek bile zordur. Bu yüzden yine bir modele dönüştürülür:

- Sol tarafta ülkeler, sağ tarafta pilotlar vardır.
- Bir pilot bir ülkeye uçmak istiyorsa aralarına bir kenar çizilir. Örneğin Libya’ya P_2, P_3 ve P_6 uçmak istiyorsa Libya bu üç pilota bağlanır.
- Aynı kümenin elemanları arasında hiçbir kenar yoktur: İki ülke arasında ya da iki pilot arasında bağlantı olamaz.

Hoca, bu yapının ileride görülecek **iki parçalı graf (bipartite graph)** olduğunu söyledi: Düğümler iki kümeye ayrılır ve kenarlar yalnız iki küme arasında olur.

Kısıtlar şunlardır:

- Bu listeye göre bir görevlendirme mümkünse bütün pilotların isteklerine uyulur.
- Mümkün değilse olabildiğince çok eşleme yapılır. Herkesin istediği yapılırsa belki Macaristan’a hiç uçuş olmayacaktır. O zaman olabildiğince çok kişi memnun edilmeye çalışılır.

Bu da bir optimizasyon problemi olarak düşünülebilir: Uçuşlar ve pilotlar, pilotların isteğine olabildiğince uyacak şekilde eşlenecek. Slaytta örnek bir eşleştirme gösterildi ve kaç pilotun isteğinin yerine geldiği sayıldı. Amaç, bütün kombinasyonlara bakıp en iyi eşleştirmeyi bulmaktır. Hoca, bu problemi farklı kişilerin farklı algoritmalarla çözebileceğini, amacın en çok isteği karşılamak olduğunu söyledi.

Slaytta bu problemle ilgili başka sorular da vardı: Olası atama sayısı kaçtır? Bütün olası atamalar, hiçbiri unutulmadan nasıl üretilir? Örneğin bir pilotun ilk isteği karşılanamıyorsa ikinci tercihi değerlendirilebilir. Bunlar algoritmik olarak çözülecek problemlerdir.

> **Ek bilgi (derste anlatılmadı):** İki parçalı bir grafta en çok sayıda kenarı ortak düğüm kullanmadan seçme problemine “en büyük eşleştirme” (maximum matching) denir. Bütün ülkelere pilot atanıp atanamayacağı Hall teoremiyle kontrol edilebilir: Ülkelerin her alt kümesi için, o ülkelere uçmak isteyen pilot sayısı alt kümedeki ülke sayısından az olmamalıdır.

10. Mantık ve Önermeler

10.1 Mantık nedir?

İkinci konu mantık ve ispatlar (logic and proofs) oldu. Hoca önce gündelik dildeki “mantıklı konuşmuyorsun” ifadesini sordu: Bu mantıkla derste anlatılan mantığın ilgisi var mı? Mantık, doğru çıkarım elde etmeye çalışmaktır. Doğru olan bir şeyi yapmak mantıklıdır; absürt bir şey yapıldığında “mantıksız” denir.

Mantığın kullanıldığı yerler:

- **Matematikte:** teoremleri ispatlamakta.
- **Bilgisayar bilimlerinde:** programların kendilerinden beklenen sonucu üretip üretmediğinin kontrolünde.

Soru (hoca sordu): Programlamada mantık nerede kullanılır? **Cevap:** Bir öğrenci önce “bilgisayar donanımlarında” dedi; hoca programlama dillerini kastettiğini söyledi. Sonra bir öğrenci

"`if` kontrolünde" dedi. Hoca bunu onayladı: `if` tamamen mantığın kullanıldığı bir yapıdır, bir şart vardır. `case`, `while`, `repeat` yapılarının hepsinde de mantık vardır.

10.2 Önerme

Önerme (proposition): Sonucu doğru ya da yanlış olan ifadeye önerme denir. Bir ifadenin önerme olabilmesi için sonucu ya doğru ya yanlış olmalıdır.

Hoca bunu öğrencilere sorular sorarak gösterdi:

- "Saat kaç?" sorusuna öğrenci "10.41" diye cevap verdi. Cevap doğru ya da yanlış değil, bir değerdir. Hocaya göre bu bir önerme değildir.
- "Saatin var mı?" sorusuna öğrenci "evet" dedi. Hoca, cevabın doğru (true) olduğunu, bu yüzden bunun bir önerme olduğunu söyledi.
- "X programcıdır." Bu bir önermedir: Ya programcıdır ya değildir.
- "Keşke bilge bir kişi olsaydım." Bu bir önerme değildir; cevabı evet ya da hayır değil, bir temennidir.

  Ders sonrası not: Hoca "Saatin var mı?" sorusunu önerme saydı. Rosen'in tanımına göre önerme, doğru ya da yanlış olan bir bildirme cümlesidir (declarative sentence). Soru, emir ve dilek cümleleri önerme değildir. "Saatin var mı?" bir önerme değildir; "Benim saatim var" ise bir önermedir.

Neden cümleler üzerinden çalışılıyor? Hoca ileride doğal dilde yazılmış bir paragraf vereceğini, bu paragrafın önermelere ayrılıp mantıksal ifadelerle yazılacağını söyledi. Yani bir metin p, q, r gibi önerme değişkenleriyle mantıksal bir ifadeye çevrilebilir.

10.3 Önerme değişkenleri ve i, j, k sorusu

Önermeler gelişigüzel a, b, c ya da x, y, z ile değil, p, q, r, s, t ile gösterilir. p harfi *proposition* kelimesinden gelir; ardından alfabedeki sonraki harfler kullanılır.

Hoca buna benzer bir alışkanlığı sordu: Algoritmalarda döngü değişkenlerine neden hep i, j, k adı verilir? Bir öğrenci "indeks olduğu için mi?" dedi. Hoca "hayır" dedi ve cevabı verdi: İlk programlama dillerinden biri olan Fortran'da (hocanın lisansta ilk öğrendiği dil) I, J, K, L gibi değişkenler değişken bölümünde tanımlanmasa bile otomatik olarak integer kabul edilirdi. Fortran çok fazla hesaplama için kullanılan bir dildi ve bu harfler çok sık kullanılıyordu; dili geliştirenler bunu uygun görmüştü. Bu değişkenler real ya da başka bir tip olacaksa ayrıca tanımlanması gerekiyordu. Döngülerde hep integer değişken kullanıldığı için bu bir alışkanlık haline geldi.

Hoca, bunun zorunlu (must) olmadığını, ama bir `for` döngüsündeki sayacın integer olmasının zorunlu olduğunu ve programcının bunu bilmesi gerektiğini ekledi.

> **Ders sonrası not:** Hoca "I, J, K, L otomatik integer" dedi. Fortran'ın örtük tip kuralında adı I, J, K, L, M, N harflerinden biriyle başlayan değişkenler tanımlanmasa da `INTEGER`, diğer harflerle başlayanlar `REAL` kabul edilir. Yani kural yalnız tek harfli değişkenler için değil, bu harflerle başlayan bütün adlar için geçerlidir.

11. Bağlaçlar ve Gösterimleri

Elimizde p, q, r gibi önermeler var. Bunlar birbirine bağlanabilir. Hocanın örneği, kendisi hakkında iki önermeydi: "Hoca Bilgisayar Mühendisliğinde öğretim üyesidir" ve "Hoca Ayrık Matematik dersini veriyor." Bunlar "ve" ile bağlanırsa: "Hoca Bilgisayar Mühendisliğinde öğretim üyesidir ve Ayrık Matematik dersini veriyor."

Önermeleri bağlayan bağlaçlar (connectives):

Bağlaç	İngilizce adı	Sembol
Ve	AND, conjunction	$\wedge$
Veya	OR, inclusive disjunction	$\vee$
Dışlayan veya	XOR, exclusive disjunction	$\oplus$

Bağlaç	İngilizce adı	Sembol
Değil	NOT, negation	$\neg p$, p', $\bar{p}$
İse	implication (conditional)	$\rightarrow$
Ancak ve ancak	double implication (biconditional)	$\leftrightarrow$

Gösterimle ilgili hocanın açıklamaları:

- AND sembolü ters çevrilmiş V harfidir ($\wedge$), OR sembolü V harfidir ($\vee$).
- XOR'un başında bir X vardır. Kitaplarda ve lojik dersinde çoğunlukla daire içinde artı işaretiyle ($\oplus$) gösterilir.
- Değilleme (negation) bir önermenin tersini almaktır. Kitaplar bunun için çok farklı işaretler kullanır: $\neg p$, $\sim p$, p', $\bar{p}$ gibi. Hocaya göre en çok kullanılanlar $\neg$ ve kesme (apostrof) işaretidir.
- İse tek yönlü bir oktur, ancak ve ancak çift yönlü bir oktur.

12. AND (Ve) ve Doğruluk Tablosunun Kuruluşu

12.1 AND çarpmadır

Hoca AND işlemini çarpmaya benzetti. Ama bu, 2×3 gibi bir sayı çarpması değildir; 0 ve 1 değerleri üzerinde yapılan bir çarpmadır. 1 doğru (true, T), 0 yanlış (false, F) demektir.

- $1 \cdot 1 = 1$
- $1 \cdot 0 = 0$ (çarpmada yutan eleman 0'dır)
- $0 \cdot 1 = 0$
- $0 \cdot 0 = 0$

Kısaca: **AND işlemi, girişlerin ikisi de doğru olduğunda doğru, diğer bütün durumlarda yanlış sonuç üretir.**

12.2 Kaç satır?

İki değişken için kaç farklı durum yazılabilir? n değişken için 2^n durum vardır. $n = 2$ için $2^2 = 4$ durum: ikisi de yok, ikisi de var, biri var biri yok, biri yok biri var.

12.3 Satırların sırası

Satırların sırası gelişigüzel değildir. Hoca tabloyu tahtada şöyle kurdu: En sağdaki sütun (burada q) birer birer değişir: $0, 1, 0, 1$. Çünkü bu sütun $2^0 = 1$ basamağına karşılık gelir. Soldaki sütun (burada p) ikişer ikişer değişir: $0, 0, 1, 1$. Çünkü $2^1 = 2$ basamağıdır. Üçüncü bir değişken olsaydı dörder dörder ($2^2 = 4$), dördüncü bir değişken olsaydı sekizer sekizer ($2^3 = 8$) değişirdi.

Bu düzenin ikinci bir faydası vardır: Her satır, ikilik (binary) bir sayıdır ve onluk (decimal) karşılığı satır numarasını verir. Hoca bunu tek tek hesapladı:

p (2^1)	q (2^0)	Onluk karşılığı
0	0	$0 \cdot 2 + 0 \cdot 1 = 0$
0	1	$0 \cdot 2 + 1 \cdot 1 = 1$
1	0	$1 \cdot 2 + 0 \cdot 1 = 2$
1	1	$1 \cdot 2 + 1 \cdot 1 = 3$

Böylece satırlar 0'dan $2^n - 1$'e kadar sayılar olarak sıralanmış olur. Slayttaki tablo bunun tersi sırayla (en üstte $1, 1$) verilmişti. Hoca bunun hiç önemli olmadığını söyledi: İki değişken için yazılabilecek bütün durumlar tabloda vardır.

> **Ders sonrası not:** Hoca onluk karşılığı hesaplarken bir yerde "2 üzeri 2, 1'di; 1 ile çarpsam 2" dedi. Kastedilen $2^1 = 2$'dir; $1 \cdot 2^1 = 2$.

12.4 AND doğruluk tablosu

p	q	$p \wedge q$
1	1	1
1	0	0
0	1	0
0	0	0

Bu noktada 10 dakikalık bir mola verildi.

12.5 Moladan sonra kısa tekrar

Moladan sonra hoca birkaç öğrenciye soru sorarak konuları tekrar ettirdi. Sorular ve cevaplar 18. bölümdeki örnek sorular arasında verilmiştir. Bir öğrenci de ilk slayttaki portrenin Newton değil George Washington olduğunu söyledi.

13. OR (Veya)

OR işlemi $\vee$ ile gösterilir. Hoca OR'u toplamaya benzetti. Ama bu, $1 + 2 = 3$ gibi bir toplama değildir; 0 ve 1 üzerinde mantıksal (Boolean) bir toplamadır. 1 ile 1 toplandığında sonuca 2 yazılmaz. Çünkü mantık olarak "bir önerme doğruysa veya diğeri doğruysa sonuç doğrudur."

- Doğru veya doğru: doğru.
- Doğru veya yanlış: doğru.
- Yanlış veya doğru: doğru.
- Yanlış veya yanlış: yanlış.

OR'da yanlış sonuç veren tek bir durum vardır: **Her iki giriş de yanlış ise sonuç yanlıştır. Diğer bütün durumlarda doğrudur.** AND'de ise durum tersineydi: İkisi de doğruysa doğru, diğerleri yanlış.

p	q	$p \vee q$
1	1	1
1	0	1
0	1	1
0	0	0

13.1 Örnek: programcı ve avukat

Önermeler şöyle olsun:

- p: "Cüneyt programcıdır."
- q: "Zeynep avukattır."

Verilen önermeler olduğu gibi doğru kabul edilir: $p = 1$, $q = 1$.

"Cüneyt programcıdır veya Zeynep avukattır" ifadesi $p \vee q$'dur. $1 \vee 1 = 1$, yani ifade doğrudur.

Bu ifadenin yanlış olması için nasıl yazılması gerekirdi? "Cüneyt programcı değildir veya Zeynep avukat değildir." Bu $\neg p \vee \neg q$'dur: $0 \vee 0 = 0$, yanlış.

Hocanın vurguladığı ilke: Birleştirilmiş bir önermenin doğru mu yanlış mı olduğuna, aradaki bağlacın doğruluk tablosuna bakarak karar verilir. Başka bir karar yolu yoktur.

14. XOR (Dışlayan Veya)

Hoca XOR'u (exclusive disjunction, "exor") çok sevdiğini söyledi, çünkü XOR ile farklı işler çok rahat yapılabilir. Kaynaklarda $\oplus$ işaretiyle de gösterilir.

XOR'da girişlerin ikisi aynıysa çıkış yanlış, farklıysa doğrudur. Doğru-doğru için sonuç yanlış, yanlış-yanlış için sonuç yanlış; diğer iki durumda doğru.

p	q	$p \oplus q$
1	1	0
1	0	1
0	1	1
0	0	0

14.1 XOR ve Assembly: Çıkarma gibi düşünmek

Hoca XOR tablosunu akılda tutmanın yolunu Assembly dili üzerinden anlattı. Assembly alçak seviyeli bir dildir: Normal kullandığımız programlama dillerinin bir alt seviyesidir; onun da altında, her şeyin 1 ve 0 ile ifade edildiği makine dili vardır. Assembly zevklidir, ama kodlaması zahmetlidir. Yüksek seviyeli bir dilde tek satırda yazılan bir okuma işlemi Assembly'de belki 10 satır tutar. Karşılığında, o tek satırın arkasında neler olduğu görülür.

Matematikte A'dan A çıkarılırsa sonuç 0'dır. Assembly'de bir yazmacı (register) sıfırlamanın yollarından biri budur:

```
SUB A, A   ; A'dan A'yı çıkar, sonuç 0
```

Bir değişkene değer atamak için `MOV` komutu da kullanılabilir. Hoca bu komutun "MOVE" diye E ile yazılmaması gerektiğini, komutun `MOV` olduğunu hatırlattı. Assembly'de değişken yerine yazmaçlar vardır; örneğin 16 bitlik `AX`, `BX`, `CX`, `DX`. Hoca örnekte basitlik için yalnız `A` dedi.

XOR da aslında aynı işi yapar:

```
XOR A, A   ; A ile A'yı XOR'la, sonuç 0
```

Çünkü iki giriş aynıdır. Doğrudan doğruyu "çıkarınca" yanlış (0), yanlıştan yanlışı çıkarınca yine yanlış (0) elde edilir.

> **Hocanın vurgusu:** XOR deyince aklınıza çıkarma gelsin. Çıkarmada iki değer aynıysa sonuç sıfırdır. XOR'da da girişler aynıysa sonuç 0, farklıysa 1'dir. Tabloyu böyle rahatça kurarsınız. Bu, ileride Assembly ve lojik derslerinde de işinize yarayacak.
>
> **Ders sonrası not:** "XOR çıkarmadır" bir hatırlatma yöntemidir. XOR aslında mod 2 toplamadır: $1 \oplus 1 = 0$, çünkü $1 + 1 = 2 \equiv 0 \pmod 2$. Mod 2'de toplama ile çıkarma aynı sonucu verdiği için hatırlatma tabloyu doğru verir. Ancak $0 - 1$ işlemi sıradan çıkarmada -1 olur; XOR'da ise $0 \oplus 1 = 1$'dir.

14.2 Örnek: "ne … ne de"

Yine p: "Cüneyt programcıdır", q: "Zeynep avukattır", $p = q = 1$.

- "Ne Cüneyt programcıdır ne de Zeynep avukattır." "Ne … ne de" yapısı iki önermeyi de olumsuz yapar. İki taraftan da yanlış gelir. Bu iki değer XOR'lanırsa $\neg p \oplus \neg q = 0 \oplus 0 = 0$; sonuç yanlıştır.
- "Cüneyt programcıdır ama Zeynep avukat değildir." Bir taraf doğru, öbürü yanlıştır: $p \oplus \neg q = 1 \oplus 0 = 1$; sonuç doğrudur.

14.3 Önermeler olumlu tanımlanır

Hoca burada "çok önemli" diyerek bir yazım ilkesini vurguladı: Önermeler her zaman olumlu biçimde verilir. Yazarken olumsuzluk işareti kullanılabilir, yani önermenin tersi (inversi) alınabilir. Doğal dilde ise "ne o ne de o" ya da "filanca şudur ama bu değildir" gibi yapılarla olumsuzluk ifade edilir.

> **Ders sonrası not:** Hoca "Bir önerme hiçbir zaman negatif verilemez" dedi. Bu, bir kitap kuralı değil, bir yazım alışkanlığıdır. "Cüneyt programcı değildir" de doğru ya da yanlış olabilen geçerli bir önermedir ve tersi alınabilir ("Cüneyt programcıdır"). Derste kastedilen şudur: Önerme değişkenleri olumlu cümlelerle tanımlanır, olumsuzluk $\neg$ ile gösterilir.

15. NOT (Değil)

Değilleme (negation, "inverse alma") $\neg$, $\sim$ ya da kesme işaretiyle gösterilir. Adı üstünde: Ne girerse tersi çıkar. Doğru ise yanlış, yanlış ise doğru. Başka bir inceliği yoktur.

p	$\neg p$
1	0
0	1

Örnek: p: "Cüneyt programcıdır." $\neg p$: "Cüneyt programcı değildir."

16. Birleştirilmiş Önermeler

Önermeler AND, OR ve NOT ile bağlanabildiği gibi daha karmaşık ifadeler de kurulabilir. Burada parantezler çok önemlidir. Hocanın gösterdiği örnek yapılar:

- $(p \vee q) \wedge r$: Önce parantez içindeki OR yapılır, sonra sonuç r ile AND'lenir.
- $p \vee (q \wedge r)$: Önce AND yapılır, sonra sonuç p ile OR'lanır.
- $(\neg p \vee \neg q) \wedge \neg r$ benzeri bir yapı: p ve q'nun tersleri alınıp OR'lanır, r'nin tersi alınır ve ikisi AND'lenir.

İki önerme şart değildir; üç, dört, beş ya da daha çok önerme olabilir. Ama önerme sayısı arttıkça ifade karmaşıklaşır.

16.1 Üç önerme için tablo

p, q, r olmak üzere 3 önerme vardır. $2^3 = 8$ farklı durum oluşur. Bu durumlar 0'dan 7'ye kadar ikilik sayılardır: 000 onlukta 0, 111 onlukta 7'dir. Sütunların değişimi:

- r (2^0): birer birer değişir.
- q (2^1): ikişer ikişer değişir.
- p (2^2): dörder dörder değişir.

Dördüncü bir değişken olsaydı $2^3 = 8$'er satır değişecekti. Slayttaki tablo yine ters sırayla (en üstte 7, yani TTT) verilmişti.

16.2 $(p \vee q) \wedge r$ doğruluk tablosu

p	q	r	$p \vee q$	$(p \vee q) \wedge r$
T	T	T	T	T
T	T	F	T	F
T	F	T	T	T
T	F	F	T	F
F	T	T	T	T
F	T	F	T	F
F	F	T	F	F
F	F	F	F	F

Hoca iki satırı adım adım hesapladı:

- $p = T$, $q = F$, $r = F$ satırı: $p \vee q = 1 + 0 = 1$. Sonra $1 \wedge r = 1 \cdot 0 = 0$. Sonuç yanlış.
- $p = F$, $q = F$, $r = T$ satırı: $p \vee q = 0 + 0 = 0$. Sonra $0 \wedge 1 = 0 \cdot 1 = 0$. Sonuç yanlış.

16.3 Tabloyu kontrol etmek ve günlük örnekler

Hoca sınavda şöyle bir soru tipi gelebileceğini söyledi: Bir birleştirilmiş ifade ve sonuç sütunu verilir, sonuçlardan biri bilerek yanlış yazılmıştır. Önermelerin aldığı değerlere göre sonuçların

doğru olup olmadığı kontrol edilir. Önermeler de gerçek cümleler olabilir: p "bugün cuma", q "Ayrık Matematik dersindeyiz" gibi.

Hoca önce bir ders ön koşulu örneği denedi: Bir dersten geçen kişi bir sonraki dersi alabilir; birinci önerme "dersten geçmek", ikinci önerme "sonraki dersi almak" olsun. Hoca bunun iyi bir örnek olmadığını, çünkü ikincinin değerinin sonucu doğrudan etkilediğini söyleyip ikinci bir örneğe geçti: yağmur ve şemsiye (çözümü 18. bölümde).

Soru: "Veya"yı doğal cümlelerde kullanabiliyoruz. XOR'u da cümle içinden anlayabilir miyiz, yoksa bunun XOR olduğu ayrıca mı söylenir? ("Ne … ne de avukattır" örneğinde olduğu gibi.)
Cevap: Doğal dilde XOR pek karşınıza çıkmayabilir. Daha çok "ise" yapıları çıkar: "Şu şöyle olursa, şu da şöyle olursa ben şunu yaparım" gibi.

17. Koşullu Önerme

17.1 Tanım

Koşullu önerme (conditional proposition) "p ise q" ya da "if p then q" diye okunur ve $p \to q$ ile gösterilir.

> **Ders sonrası not:** Hoca bu kısma girerken "logical equivalence diyoruz, conditional proposition diye de geçer" dedi. Bunlar ayrı kavramlardır. $p \to q$ koşullu önermedir (conditional). Mantıksal denklik (logical equivalence), iki birleştirilmiş önermenin bütün durumlarda aynı doğruluk değerini alması demektir ve bir sonraki dersin konusudur.

Hoca bir uyarı yaptı: Bu, programlama dillerindeki "eğer p ise şunu yap" gibi düşünülmemelidir. Mantıktaki `if-then` yapısının kendine özgü bir doğruluk tablosu vardır ve sonuç o tabloya göre belirlenir.

p'ye hipotez (hypothesis), q'ya sonuç (conclusion) denir. Bazı kitaplar "if p then q" yerine "p only if q" yazar; ikisi aynı koşullu önermeyi ifade eder.

17.2 Doğruluk tablosu

Hoca bu tablonun "çok enteresan ve değişik" olduğunu ve bir şekilde akılda tutulması gerektiğini söyledi. Sonuçların neredeyse hepsi doğrudur; yalnız bir durumda yanlıştır. O yüzden tabloyu yanlış olan tek durumla hatırlamak en kolayıdır:

$p \to q$ **yalnız** p **doğru ve** q **yanlış iken yanlıştır. Diğer bütün durumlarda doğrudur.**

p	q	$p \to q$
1	1	1
1	0	0
0	1	1
0	0	1

Yani bir cümlede ya da birleştirilmiş ifadede p olduğu gibi (doğru) kalmış, q ise tersine çevrilerek verilmişse, "p ise $\neg q$" ifadesinin sonucu yanlıştır.

> **Ek bilgi (derste anlatılmadı):** p yanlışken $p \to q$'nun doğru sayılmasına "boş yere doğruluk" (vacuous truth) denir. Yaygın bir sezgi bir sözdür: "Sınavdan 100 alırsan sana araba alırım." Öğrenci 100 alamazsa, araba alınsa da alınmasa da söz bozulmuş olmaz. Söz yalnız öğrenci 100 alıp araba alınmazsa, yani $1 \to 0$ durumunda bozulur.

17.3 Örnek

Yine p: "Cüneyt programcıdır", q: "Zeynep avukattır", $p = q = 1$.

- "Eğer Cüneyt programcıysa Zeynep avukattır." Bu $p \to q$'dur. İki önerme de değiştirilmemiş: $1 \to 1 = 1$. İfade doğrudur.
- İfadenin yanlış çıkması için ne denmeliydi? "Eğer Cüneyt programcıysa Zeynep avukat değildir." Bu $p \to \neg q$'dur: $1 \to 0 = 0$. İfade yanlıştır.

17.4 Dersin sonundaki kısa özet

Hoca dersi şu özetle bağladı:

- **AND:** Çarpma. Girişlerin ikisi de doğruysa sonuç doğru ($1 \cdot 1 = 1$).
- **OR:** Toplama. En az bir giriş doğruysa sonuç doğru ($1, 1 \to 1$; $1, 0 \to 1$; $0, 1 \to 1$).
- **NOT:** Ne giriyorsa tersi. 0 ise 1, 1 ise 0.
- **XOR:** Çıkarma. Girişler aynıysa sonuç 0; doğru-doğru yanlış, yanlış-yanlış yanlış, diğerleri doğru.
- **İse:** Yanlış veren tek durum doğru-yanlıştır.

Hoca bir sonraki derse mantıksal denklik (logical equivalence) ile başlayacağını söyledi ve bu konulara bakarak gelinmesini istedi.

18. Örnek Sorular

Bu derste ekran görüntüsü kullanılmadı. Aşağıdaki sorular derste sözlü olarak sorulan ya da slayttan okunan sorulardır.

Soru 1 (derste sözlü, 01:16:29)

Altı kişilik bir yatırım kulübü her yıl başkan ve sayman pozisyonlarını dönüşümlü değiştirmek istiyor. Aynı kişilerin aynı pozisyonlara yeniden gelmesi için kaç yıl geçmelidir?

> **Çözüm (derste verilmedi):** Başkan 6 kişiden biri olabilir, sayman kalan 5 kişiden biri. Sıra önemlidir (başkan ve sayman farklı görevlerdir), bu yüzden farklı atama sayısı $P(6,2) = 6 \cdot 5 = 30$'dur. Her yıl farklı bir atama yapılırsa 30 yıl boyunca hiçbir atama tekrarlanmaz. 31. yılda ilk kez daha önce kullanılmış bir atamaya dönülmek zorundadır; yani en çok 30 yıl geçtikten sonra aynı ikili aynı pozisyonlara gelir. Sonuç Python ile bütün sıralı ikililer sayılarak doğrulandı (30).

Soru 2 (derste sözlü, 01:15:38)

Dört evli çift, iki kortta karışık çiftler tenis oynuyor. İki saat boyunca her yarım saatte (yani 4 tur) eşlerini ve rakiplerini değiştiriyorlar. Her erkeğin her kadınla en az bir kez birlikte ve en az bir kez karşı karşıya oynadığı, ayrıca her erkeğin diğer her erkeğe karşı en az bir kez oynadığı bir fikstür var mı?

> **Çözüm (derste verilmedi):** Derste bunun zor bir varlık problemi olduğu söylendi, çözülmedi. Soru yukarıdaki gibi okunursa (4 tur, her turda iki kortta erkek-kadın çiftleri karşılaşıyor), bütün olası fikstürler Python ile iki ayrı yoldan tek tek denendi ve üç koşulu birlikte sağlayan fikstür bulunamadı. Koşulların her biri tek başına sağlanabiliyor, ama birlikte sağlanamıyor. Bu yüzden bu okumayla cevap "hayır"dır. Kitaptaki asıl soru farklı bir tur sayısı ya da farklı bir koşul içeriyorsa sonuç değişebilir.

Soru 3 (derste sözlü, 01:19:14)

Bir mağazanın ilan hazırlığı için A'dan K'ya 11 iş var. Süreler ve öncelikler 8. bölümdeki tablodadır. Arka arkaya yapılınca 37 gün süren işler, 30 günü aşmadan nasıl tamamlanır?

Çözüm derste verildi (8.3): Her iş, önceki işlerin en geç bitişine kendi süresi eklenerek biter. Bütün işler 28 günde tamamlanır. Kritik yol $A \to C \to D \to F \to I \to J \to K$'dir. Sonuçlar Python ile doğrulandı.

Soru 4 (derste sözlü, 01:40:32)

Bilgisayar bilimlerinde, özellikle programlamada mantık nerede kullanılır?

Cevap derste verildi: `if` yapısında. `if` tamamen mantığın kullanıldığı bir yapıdır. `case`, `while`, `repeat` yapılarında da mantık vardır.

Soru 5 (derste sözlü, 01:42:07)

Aşağıdakiler önerme midir?

- "Saat kaç?"
- "Saatin var mı?"
- "X programcıdır."
- "Keşke bilge bir kişi olsaydım."

Derste verilen cevap: "Saat kaç?" önerme değildir, çünkü cevabı doğru ya da yanlış değildir. "Saatin var mı?" sorusu, cevabı evet olduğu için önerme sayıldı. "X programcıdır" önermedir. "Keşke bilge bir kişi olsaydım" bir temennidir, önerme değildir.

> **Ders sonrası not:** Rosen'in tanımına göre soru cümleleri önerme değildir. "Saatin var mı?" önerme değildir. Önermeler yalnız "X programcıdır" gibi bildirme cümleleridir.

Soru 6 (derste sözlü, 01:44:01)

Algoritmalarda döngü değişkenlerine neden çoğunlukla i, j, k adı verilir?

Cevap derste verildi: Bu bir indeks kısaltması değil, Fortran'dan kalan bir alışkanlıktır. Fortran'da bu harflerle adlandırılan değişkenler tanımlanmasa bile integer kabul edilirdi. Döngü sayaçları integer olduğu için bu adlar yaygınlaştı (ayrıntı ve düzeltme 10.3'te).

Soru 7 (derste sözlü, 02:10:19)

Moladan sonraki tekrar soruları:

- Matematikle ayrık matematik arasındaki fark nedir? Ayrık matematik bize ne sağlar?
- Önerme (proposition) nedir?
- Önermeleri birbirine nelerle bağlarız?
- Ayrık matematiğin konu başlıklarından aklınızda neler kaldı?

Derste verilen cevaplar: Ayrık matematikte sürekli olmayan, ayrık yapılarla çalışılır. Sürekli problemler, örnekler alınarak ayrık hale getirilir ve böylece çözülebilir. Önerme, doğru ya da yanlış diyebileceğimiz ifadedir. Önermeler AND, OR, XOR ve diğer bağlaçlarla bağlanır. Konu başlıklarından en kısa yol örnekleri, graflar ve ağaçlar hatırlandı. Okunan konunun adı mantık ve ispatlardır (logic and proofs).

Soru 8 (derste sözlü, 02:18:28)

p: "Cüneyt programcıdır", q: "Zeynep avukattır" ve ikisi de doğru olsun. Aşağıdaki ifadelerin doğruluk değeri nedir?

- "Cüneyt programcıdır veya Zeynep avukattır."
- "Cüneyt programcı değildir veya Zeynep avukat değildir."
- "Ne Cüneyt programcıdır ne de Zeynep avukattır." (XOR ile)
- "Cüneyt programcıdır ama Zeynep avukat değildir." (XOR ile)
- "Eğer Cüneyt programcıysa Zeynep avukattır."
- "Eğer Cüneyt programcıysa Zeynep avukat değildir."

Çözüm derste verildi:

İfade	Hesap	Sonuç
$p \vee q$	$1 \vee 1$	Doğru
$\neg p \vee \neg q$	$0 \vee 0$	Yanlış
$\neg p \oplus \neg q$	$0 \oplus 0$	Yanlış
$p \oplus \neg q$	$1 \oplus 0$	Doğru
$p \to q$	$1 \to 1$	Doğru
$p \to \neg q$	$1 \to 0$	Yanlış

Bütün değerler Python ile doğrulandı.

Soru 9 (derste sözlü, 02:32:38)

$(p \vee q) \wedge r$ ifadesinin doğruluk tablosunu kurun. $p = T, q = F, r = F$ ve $p = F, q = F, r = T$ satırlarında sonuç nedir?

Çözüm derste verildi (16.2): İlk satırda $(1 + 0) \cdot 0 = 0$, ikinci satırda $(0 + 0) \cdot 1 = 0$; ikisi de yanlıştır. Tablonun tamamı Python ile doğrulandı.

Soru 10 (derste sözlü, 02:35:48)

"Bugün hava yağmurlu" ve "şemsiyem var" önermelerine göre ıslanır mıyım? Derste dört durum sözlü olarak verildi:

- Hava yağmurlu, şemsiyem var: ıslanmam.
- Hava yağmurlu, şemsiyem yok: ıslanırım.
- Hava yağmurlu değil, şemsiyem var: ıslanmam.
- Hava yağmurlu değil, şemsiyem yok: ıslanmam.

Hoca "böyle bir durum varsa bu ifadeyi matematiksel olarak nasıl kodlarsın?" diye sordu ve bunun ileriki slaytlarda geleceğini söyledi.

> **Çözüm (derste verilmedi):** y: "hava yağmurlu", s: "şemsiyem var" olsun. Islanma yalnız $y = 1$, $s = 0$ iken doğrudur. Bu, $y \wedge \neg s$ ifadesidir.
>
> | y | s | $\neg s$ | $y \wedge \neg s$ (ıslanırım) |
> |---|---|---|---|
> | 1 | 1 | 0 | 0 |
> | 1 | 0 | 1 | 1 |
> | 0 | 1 | 0 | 0 |
> | 0 | 0 | 1 | 0 |
>
> Tablo Python ile doğrulandı.

Dersin Toparlaması

Bu ders, ayrık matematiğin ne olduğunu ve neden bir bilgisayar mühendisi için temel olduğunu anlatarak başladı. Bilgisayar sürekli verilerle çalışamaz; sürekli bir işaretten örnekler alınır ve işlem bu ayrık örnekler üzerinde yapılır. Ayrık matematik bu yüzden süreklilik içermeyen yapılarla ilgilenir: tam sayılar, program adımları, bir yolun durakları. Şifre sayma, en kısa yol, eşleştirme ve planlama gibi problemler, metinden bir modele, çoğu zaman bir grafa çevrilerek çözülür. PERT örneğinde 37 günlük sıralı plan, öncelik grafı kurularak 28 güne indi. Pilot örneğinde istekler iki parçalı bir grafla gösterildi. Bu dönüşümü yapabilmek, hocanın sürekli vurguladığı algoritmik düşüncenin kendisidir.

Dersin ikinci yarısında mantığa girildi. Önerme, doğru ya da yanlış değeri olan ifadedir. Önermeler AND (çarpma), OR (toplama), XOR (girişler aynıysa 0), NOT (tersini alma) ve koşullu önerme ($p \to q$, yalnız $1 \to 0$ iken yanlış) ile bağlanır. n önerme için 2^n satırlık doğruluk tablosu, satırlar 0'dan $2^n - 1$'e kadar ikilik sayılar olacak şekilde kurulur. Bir sonraki derste mantıksal denklik (logical equivalence) ile devam edilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.